Lab 3 Report

과제 개요 및 구현 과정

이번 과제는 Verilog를 사용하여 Single-cycle CPU를 구현하는 과제다. 구현해야 하는 CPU는 MIPS ISA를 지원한다. Skeleton Code의 `CPU.v`, `CTRL.v`, `ALU.v`, `RF.v` 파일의 내용을 수정하였다.

`ALU.v`와 `RF.v`는 각각 Lab 1과 Lab 2에서 구현한 그대로 가져왔다.

```
case (funct)
    `ALU_ADDU: alu_result = operand1 + operand2;
    `ALU_AND : alu_result = operand1 & operand2;
    `ALU_NOR : alu_result = ~(operand1 | operand2);
    `ALU_OR  : alu_result = operand1 | operand2;
    `ALU_SLL : alu_result = operand2 << shamt;
    `ALU_SRA : alu_result = $signed(operand2) >>> shamt;
    `ALU_SRL : alu_result = operand2 >> shamt;
    `ALU_SUBU: alu_result = operand1 - operand2;
    `ALU_XOR : alu_result = operand1 ^ operand2;
    `ALU_SLT : alu_result = ($signed(operand1) < $signed(operand2)) ? 32'd1 : 32'd0; // Signed 비교
    `ALU_SLTU: alu_result = (operand1 < operand2) ? 32'd1 : 32'd0; // Unsigned 비교
    `ALU_EQ  : alu_result = (operand1 == operand2) ? 32'd1 : 32'd0;
    `ALU_NEQ : alu_result = (operand1 != operand2) ? 32'd1 : 32'd0;
    `ALU_LUI : alu_result = operand2 << 16; // LSB 16-bit에 있는 Immediate 값을 MSB 16-bit로 이동
endcase
```

```
3   module RF (
4       input clk,
5       input rst,
6       // Read-related ports
7       input [4:0] rd_addr1,
8       input [4:0] rd_addr2,
9       output reg [31:0] rd_data1,
10      output reg [31:0] rd_data2,
11      // Write-related ports
12      input RegWrite,
13      input [4:0] wr_addr,
14      input [31:0] wr_data
15  );
16
17  reg [31:0] register_file [0:31];
18
19  always @(*) begin
20      rd_data1 = register_file[rd_addr1];
21      rd_data2 = register_file[rd_addr2];
22  end
23
24  always @(posedge clk) begin
25      if (rst) begin
26          $readmemh("initial_reg.mem", register_file);
27      end
28      else begin
29          if (RegWrite) begin
30              register_file[wr_addr] <= wr_data;
31          end
32      end
33  end
34
35  endmodule
36
```

기존 과제의 경우 R-type instruction만을 지원하였지만, 이번 과제에서는 추가로 I-type instruction과 J-type instruction을 구현해야 한다. ALU에 연산자를 추가할 필요 없이 컨트롤 시그널을 설정해주는 컨트롤 유닛 `CTRL.v`에서 이를 조정해주면 된다.

```
4   module CTRL(
5       // input opcode and funct
6       input [5:0] opcode,
7       input [5:0] funct,
8
9       // output various ports
10      output reg RegDst,
11      output reg Jump,
12      output reg Branch,
13      output reg JR,
14      output reg MemRead,
15      output reg MemtoReg,
16      output reg MemWrite,
17      output reg ALUSrc,
18      output reg SignExtend,
19      output reg RegWrite,
20      output reg [3:0] ALUOp,
21      output reg SavePC
22      );
23
```

컨트롤 유닛 CTRL에서는 2개의 입력으로 명령어 유형을 구분하기 위한 `opcode`와 R-type 명령어 내부에서 연산자를 파악하기 위한 `funct`를 입력받는다. 12개의 출력으로 각각의 컨트롤 시그널을 출력한다.

- RegDst: RF에 데이터를 쓸 때, `rt`에 쓰면 0, `rd`에 쓰면 1로 설정
- Jump: Jump 명령어일 경우 1, 그 외에는 0으로 설정
- Branch: Branch 명령어일 경우 1, 그 외에는 0으로 설정
- JR: 1로 설정되어 있으면 PC를 `rs`의 값으로 덮어씀
- MemRead: 메모리에서 값을 읽어야 하면 1, 그 외에는 0으로 설정
- MemtoReg: RF에 쓰는 데이터의 출처에 따라 ALU의 연산값을 쓰면 0, 메모리에서 가져온 데이터를 쓰면 1로 설정
- MemWrite: 메모리에 값을 써야 하면 1, 그 외에는 0으로 설정
- ALUSrc: `operand2`의 값으로 RF에서 받아온 `rt`를 사용하면 0, immediate 값을 사용하면 1로 설정

- SignExtend: 16-bit immediate 값을 32-bit로 확장할 때, MSB에 0으로 채우면 0, 부호에 따라 채우면 1로 설정
- RegWrite: RF에 데이터를 쓰면 1, 쓰지 않으면 0으로 설정
- ALUOp: 4-bit로 어떤 연산을 수행할 지 결정
- SavePC: 1로 설정되어 있으면 JAL 명령어 수행 시 PC + 4 값을 `ra`(31번 레지스터)에 저장

```
24      always @(*) begin
25          RegDst = 0;
26          Jump = 0;
27          Branch = 0;
28          JR = 0;
29          MemRead = 0;
30          MemtoReg = 0;
31          MemWrite = 0;
32          ALUSrc = 0;
33          SignExtend = 0;
34          RegWrite = 0;
35          ALUOp = 4'b0000;
36          SavePC = 0;
```

먼저 컨트롤 시그널을 초기화해주었다.

```
37          case(opcode)
38              `OP_RTYPE: begin
39                  if(funct == `FUNCT_JR) begin
40                      JR = 1;
41                  end else begin
42                      RegDst = 1;
43                      RegWrite = 1;
44                      case(funct)
45                          `FUNCT_ADDU: ALUOp = `ALU_ADDU;
46                          `FUNCT_SUBU: ALUOp = `ALU_SUBU;
47                          `FUNCT_AND : ALUOp = `ALU_AND;
48                          `FUNCT_OR  : ALUOp = `ALU_OR;
49                          `FUNCT_XOR : ALUOp = `ALU_XOR;
50                          `FUNCT_NOR : ALUOp = `ALU_NOR;
51                          `FUNCT_SLT : ALUOp = `ALU_SLT;
52                          `FUNCT_SLTU: ALUOp = `ALU_SLTU;
53                          `FUNCT_SLL : ALUOp = `ALU_SLL;
54                          `FUNCT_SRL : ALUOp = `ALU_SRL;
55                          `FUNCT_SRA : ALUOp = `ALU_SRA;
56                      endcase
57                  end
58              end
```

R-type 명령어를 분기하는 코드다.

`JR` 명령어의 경우 `JR` 컨트롤 시그널을 1로 설정하여 PC를 `rs` 값으로 설정한다. 이렇게 하면 `JAL` 실행 시 함수 실행이 끝난 후 돌아가야 하는 주소로 돌아갈 수 있다.

이 외의 R-type 명령어는 모두 RF의 `rd` 레지스터에 데이터를 쓰기 때문에 `RegDst`와 `RegWrite` 모두 1로 설정하였다.

```
59              `OP_ADDIU: begin
60                  ALUSrc = 1;
61                  SignExtend = 1;
62                  RegWrite = 1;
63                  ALUOp = `ALU_ADDU;
64              end
65              `OP_ANDI: begin
66                  ALUSrc = 1;
67                  SignExtend = 1;
68                  RegWrite = 1;
69                  ALUOp = `ALU_AND;
70              end
71              `OP_LUI: begin
72                  ALUSrc = 1;
73                  SignExtend = 0;
74                  RegWrite = 1;
75                  ALUOp = `ALU_LUI;
76              end
77              `OP_ORI: begin
78                  ALUSrc = 1;
79                  SignExtend = 0;
80                  RegWrite = 1;
81                  ALUOp = `ALU_OR;
82              end
83              `OP_SLTI: begin
84                  ALUSrc = 1;
85                  SignExtend = 1;
86                  RegWrite = 1;
87                  ALUOp = `ALU_SLT;
88              end
89              `OP_SLTIU: begin
90                  ALUSrc = 1;
91                  SignExtend = 1;
92                  RegWrite = 1;
93                  ALUOp = `ALU_SLTU;
94              end
```

```
95              `OP_XORI: begin
96                  ALUSrc = 1;
97                  SignExtend = 0;
98                  RegWrite = 1;
99                  ALUOp = `ALU_XOR;
100             end
101             `OP_BEQ: begin
102                 Branch = 1;
103                 SignExtend = 1;
104                 ALUOp = `ALU_EQ;
105             end
106             `OP_BNE: begin
107                 Branch = 1;
108                 SignExtend = 1;
109                 ALUOp = `ALU_NEQ;
110             end
111             `OP_J: begin
112                 Jump = 1;
113             end
114             `OP_JAL: begin
115                 Jump = 1;
116                 RegWrite = 1;
117                 SavePC = 1; // 현재 PC 값을 저장하여 나중에 레지스터에 쓸 수 있도록 함
118             end
119             `OP_LW: begin
120                 MemRead = 1;
121                 MemtoReg = 1;
122                 ALUSrc = 1;
123                 SignExtend = 1;
124                 RegWrite = 1;
125                 ALUOp = `ALU_ADDU;
126             end
127             `OP_SW: begin
128                 MemWrite = 1;
129                 ALUSrc = 1;
130                 SignExtend = 1;
131                 ALUOp = `ALU_ADDU;
132             end
133             default: begin
134             end
135         endcase
```

I-type ALU 명령어의 경우 모두 `operand2`로 immediate 값을 사용하고, RF의 `rt` 레지스터에 데이터를 쓰기 때문에 `ALUSrc`와 `RegWrite` 모두 1로 설정하였다.

`ADDI`, `ANDI`, `SLTI`, `SLTIU` 명령어는 모두 부호를 고려하여 연산을 수행하기 때문에 `SignExtend`를 1로 설정하여 부호에 맞는 값을 채워넣어준다.

`ORI` 명령어는 논리 연산의 특성상 상수값의 부호 유지 대신 상위 16-bit를 0으로 채우는 zero-extension을 수행하고, `LUI` 명령어는 상수값을 레지스터의 상위 16-bit 위치로 직접 이동시키기 때문에 `SignExtend`를 0으로 설정하였다.

Branch 명령어는 부호를 고려하여 연산을 수행하기 때문에 `SignExtend`를 1로 설정하였다. Jump 명령어 중 `J`는 단순히 `Jump`만 1로 설정하고, `JAL`은 현재 PC 값(PC + 4)을 `ra` 레지스터에 저장하기 때문에 `RegWrite`와 `SavePC`를 1로 설정하였다.

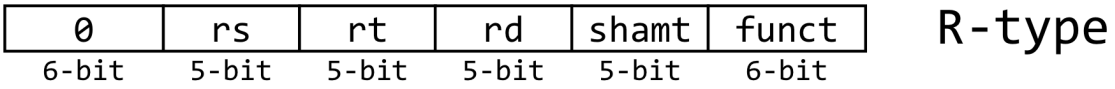
`LW` 명령어는 `rs` 레지스터에서 읽은 주소값과 immediate 값을 sign-extend한 값을 `ADD` 연산하여 메모리 주소를 계산하고, 해당 주소의 메모리에서 데이터를 읽어와 RF에 쓰기 때문에 `MemRead`, `MemtoReg`, `ALUSrc`, `SignExtend`, `RegWrite` 모두 1로 설정하였다.

`SW` 명령어는 `rs` 레지스터에서 읽은 주소값과 immediate 값을 sign-extend한 값을 `ADD` 연산하여 그 결과값의 주소가 가리키는 데이터를 메모리에 쓰기 때문에 `MemWrite`, `ALUSrc`, `SignExtend` 모두 1로 설정하였다.

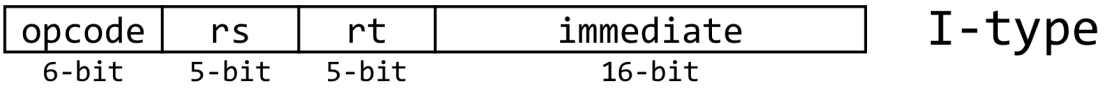
```
57      // Define PC
58      reg [31:0]  PC;
59      reg [31:0]  PC_next;
60      wire [31:0] PC_plus_4 = PC + 32'd4;
61
62      // Define the wires
63      assign opcode = inst[31:26];
64      assign rs = inst[25:21];
65      assign rt = inst[20:16];
66      assign rd = inst[15:11];
67      assign shamt = inst[10:6];
68      assign funct = inst[5:0];
69      assign immi = inst[15:0];
70      assign immj = inst[25:0];
71
72      assign ext_imm = SignExtend ? {{16{immi[15]}}, immi} : {16'b0, immi};
73
74      assign operand1 = rd_data1;
75      assign operand2 = ALUSrc ? ext_imm : rd_data2;
76
77      wire  branch_taken = Branch & alu_result[0];
78
79      assign halt = (inst == 32'b0);
```

`CPU.v`에서 PC + 4 값을 `PC_plus_4` 변수에 담았다.

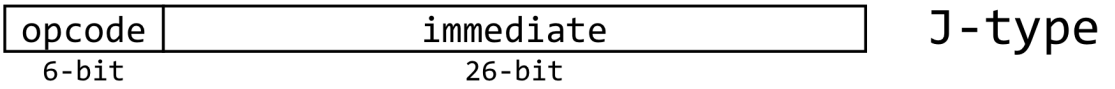
- R-type, 3 register operands



- I-type, 2 register operands and 16-bit immediate operand



- J-type, 26-bit immediate operand



- `opcode`: 명령어 유형과 관계없이 `inst`의 상위 6-bit ([31:26])
- `rs`: R-type과 I-type 명령어에서 `opcode` 다음 5-bit ([25:21])
- `rt`: R-type과 I-type 명령어에서 `rs` 다음 5-bit ([20:16])
- `rd`: R-type 명령어에서 `rt` 다음 5-bit ([15:11])
- `shamt`: R-type 명령어에서 `rd` 다음 5-bit ([10:6])
- `funct`: R-type 명령어의 하위 6-bit ([5:0])
- `immi`: I-type 명령어의 하위 16-bit ([15:0])
- `immj`: J-type 명령어의 하위 26-bit ([25:0])
- `ext_imm`: `SignExtend`가 1이면 16-bit immediate 값의 최상위 비트 `immi[15]`를 부호로 사용하여 해당 비트값을 16번 반복하여 채워넣음. `SignExtend`가 0이면 0으로 상위 16-bit를 채워넣음
- `operand2`: `ALUSrc`가 1이면 immediate 값을 사용하기 때문에 `ext_imm`, 0이면 `rt`를 사용하기 때문에 rd_data2
- `branch_taken`: 해당 명령어가 branch 명령어고, branch 명령어의 결과값인 `alu_result`의 최하위 비트(`alu_result[0]`)가 1이면 branch taken, 0이면 branch not taken임을 `&` 연산 수행

```
81      always @(*) begin
82          if (JR) begin
83              PC_next = rd_data1;
84          end else if (Jump) begin
85              PC_next = {PC_plus_4[31:28], immj, 2'b00};
86          end else if (branch_taken) begin
87              PC_next = PC_plus_4 + (ext_imm << 2);
88          end else begin
89              PC_next = PC_plus_4;
90          end
91
92          if (SavePC) begin
93              wr_addr = 5'd31;
94              wr_data = PC_plus_4;
95          end else begin
96              wr_addr = RegDst ? rd : rt;
97              wr_data = MemtoReg ? mem_read_data : alu_result;
98          end
99      end
100
101
```

1. PC값 계산
    a. `JR`이면 nextPC에 `rs` 레지스터의 값을 저장한다.
    b. Jump 명령어의 경우 nextPC를 PC + 4의 상위 4-bit, 26-bit immediate 값, 그리고 2-bit 00을 concatenate 한 값으로 설정한다.
    c. Branch 명령어의 경우 branch가 taken이면 PC + 4 + sign-extend한 label(immediate 값)을 더하여 branch target address를 계산한다.
    d. Branch 명령어인데 branch가 not taken이거나, 그 외 명령어의 경우 nextPC를 PC + 4로 설정한다.
2. Write Address/Data 설정
    a. `SavePC`가 1인 경우: `JAL` 명령어라는 의미기 때문에 31번 레지스터에 PC + 4를 써야 한다.
    b. 그 외의 경우: `RegDst`가 1이면 `rd`에 써야 하고, 0이면 `rt`에 써야 한다. `MemtoReg`가 1이면 메모리에서 읽어온 값을 써야 하고, 0이면 ALU의 연산값을 써야 한다.

```
111     CTRL ctrl (
112         .opcode(opcode),
113         .funct(funct),
114         .RegDst(RegDst),
115         .Jump(Jump),
116         .Branch(Branch),
117         .JR(JR),
118         .MemRead(MemRead),
119         .MemtoReg(MemtoReg),
120         .MemWrite(MemWrite),
121         .ALUSrc(ALUSrc),
122         .SignExtend(SignExtend),
123         .RegWrite(RegWrite),
124         .ALUOp(ALUOp),
125         .SavePC(SavePC)
126     );
127 
128     RF rf (
129         .clk(clk),
130         .rst(rst),
131         .rd_addr1(rs),
132         .rd_addr2(rt),
133         .rd_data1(rd_data1),
134         .rd_data2(rd_data2),
135         .RegWrite(RegWrite),
136         .wr_addr(wr_addr),
137         .wr_data(wr_data)
138     );
139 
140     MEM mem (
141         .clk(clk),
142         .rst(rst),
143         .inst_addr(PC),
144         .inst(inst),
145         .mem_addr(alu_result),
146         .MemWrite(MemWrite),
147         .mem_write_data(rd_data2),
148         .mem_read_data(mem_read_data)
149     );
150 
151     ALU alu (
152         .operand1(operand1),
153         .operand2(operand2),
154         .shamt(shamt),
155         .funct(ALUOp),
156         .alu_result(alu_result)
157     );
158 
```

Lab 2에서 모듈을 instantiate하고 연결하는 것과 동일하게 CTRL, RF, MEM, ALU를 연결하여 single-cycle CPU를 완성하였다.

테스트

주어진 7개의 테스트케이스를 한 번에 돌리기 위한 shell 명령어 파일 `run_all.sh`을 만들었다.

$ run_all.sh

```
#!/bin/bash

# 1. 시뮬레이션 코드 단 한 번만 컴파일
echo "Compiling Verilog files..."
iverilog -o cpu_sim CPU_tb.v CPU.v CTRL.v ALU.v RF.v MEM.v

# 컴파일 에러 발생 시 중단
if [ $? -ne 0 ]; then
    echo "ERROR: Compilation failed. Please check your Verilog code."
    exit 1
fi

# 2. Testcase 1부터 7까지 반복문 실행
for i in {1..7}
do
    echo "========================================"
    echo "          Running Testcase $i         "
    echo "========================================"

    # 정답 파일(reference) 복사해오기
    cp testcase/testcase${i}/reference_mem.mem .
    cp testcase/testcase${i}/reference_reg.mem .

    # 파이썬 스크립트로 초기(initial) 파일 생성
    python3 workload_gen.py testcase/testcase${i}/testcase${i}.hex

    # 시뮬레이션 실행
    vvp cpu_sim
done

echo "========================================"
echo "          All testcases finished!       "
echo "========================================"
```

```
yjkim0306 ~/Documents/3-1/컴퓨터구조론/lab/HW03 Skeleton Code  main bash run_all.sh
Compiling Verilog files...
========================================
          Running Testcase 1
========================================
Loading instruction memory...
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 6225000 (1ps)
========================================
          Running Testcase 2
========================================
Loading instruction memory...
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 1985000 (1ps)
========================================
          Running Testcase 3
========================================
Loading instruction memory...
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 1655000 (1ps)
========================================
          Running Testcase 4
========================================
Loading instruction memory...
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 6815000 (1ps)
========================================
          Running Testcase 5
========================================
Loading instruction memory...
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 3165000 (1ps)
========================================
          Running Testcase 6
========================================
Loading instruction memory...
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 2005000 (1ps)
========================================
          Running Testcase 7
========================================
Loading instruction memory...
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 5085000 (1ps)
========================================
          All testcases finished!
========================================
```

7개의 테스트케이스 모두 성공하였다.

결론 및 JAL/JR 명령어 구현

```
`OP_JAL: begin
    Jump = 1;
    RegWrite = 1;
    SavePC = 1; // 현재 PC 값을 저장하여 나중에 레지스터에 쓸 수 있도록 함
```

```
92          if (SavePC) begin
93              wr_addr = 5'd31;
94              wr_data = PC_plus_4;
95          end else begin
96              wr_addr = RegDst ? rd : rt;
97              wr_data = MemtoReg ? mem_read_data : alu_result;
98          end
```

MIPS 아키텍처에서 `JAL` 명령어는 Jump 한 후(주로 함수 호출 이후) 다시 돌아와야 하는 위치인 PC + 4 값을 `ra`(31번) 레지스터에 저장한다. `SavePC`, 말 그대로 PC 값을 저장하는 유일한 명령어기 때문에 이 컨트롤 변수를 사용하면 write address로 31번 레지스터를 사용할 지, `RegDst` 값에 따라 `rd` 또는 `rt` 레지스터를 사용할 지 구분할 수 있다.

`CPU.v`의 write address/data를 지정해주는 부분에서 `SavePC`를 분기 변수로 설정하여 PC + 4를 31번 레지스터에 쓰도록 하였다.

```
case(opcode)
    `OP_RTYPE: begin
        if(funct == `FUNCT_JR) begin
            JR = 1;
```

```
81      always @(*) begin
82          if (JR) begin
83              PC_next = rd_data1;
84          end else if (Jump) begin
85              PC_next = {PC_plus_4[31:28], immj, 2'b00};
```

`JR` 명령어는 R-type 명령어고 `JAL`과 함께 쓰인다. 주로 함수 호출에서 `JAL` 명령어가 함수 호출 이후 PC의 위치를 저장해놓는 역할을 하고, `JR` 명령어는 `JAL`이 저장해놓은 PC 주소로 점프하는 역할을 한다. R-type 명령어이기 때문에 따로 `GLOBAL.v`에 `funct` 값이 `6'd8`로 정의가 되어 있고, 컨트롤 시그널 `JR`을 1로 설정하면 PC에 `rs` 레지스터에 있는 값을 저장하도록 분기문을 사용하였다.